

جلسه دفاع از پروژه کارشناسی

زبان برنامه نویسی تابعی Haskell برای روش های عددی

سخنران: سید احسان حسینی

زمان:

مکان:

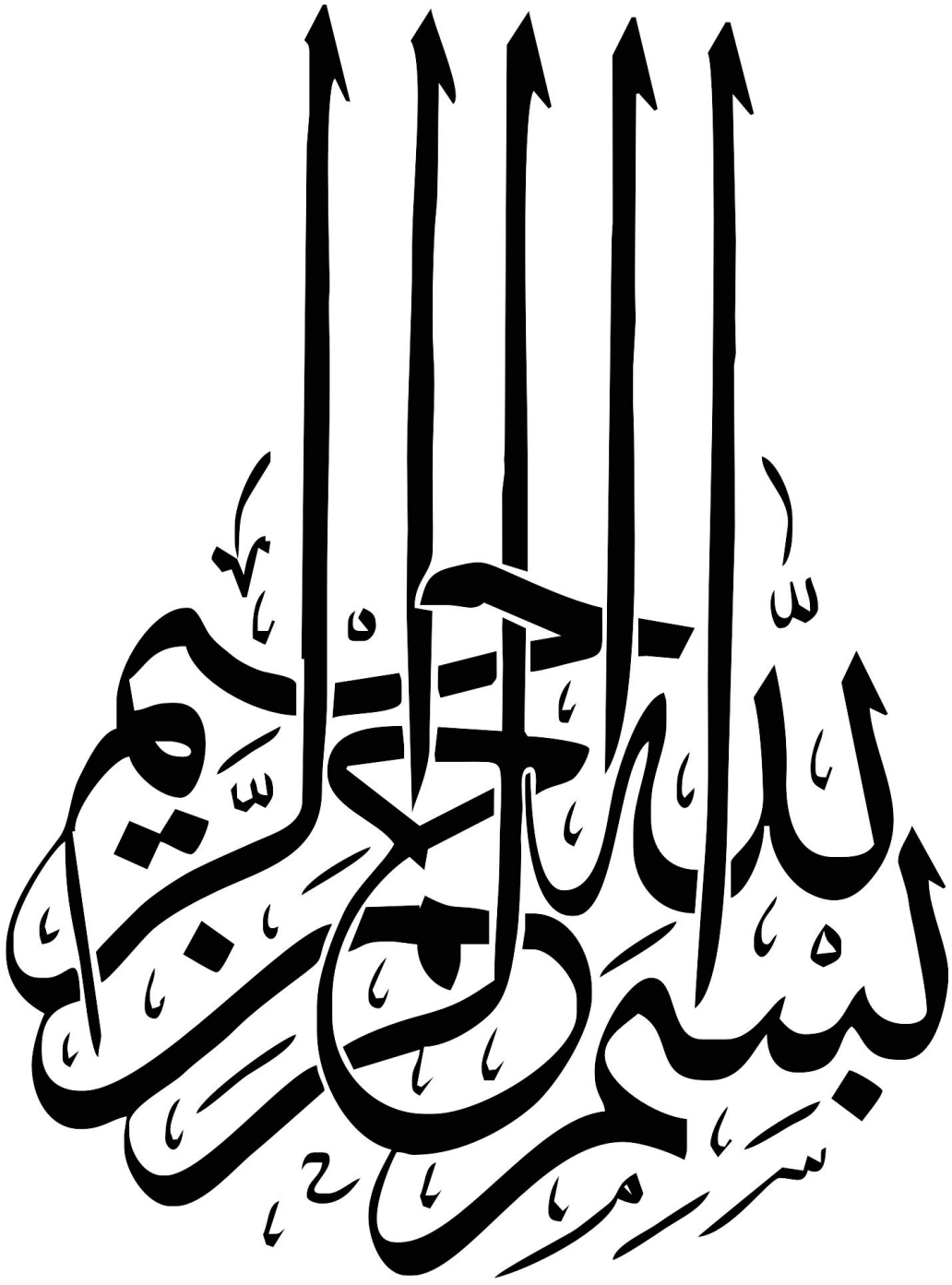
هیئت داوران

۱- دکتر رضا مختاری

۲-

۳-

چکیده:


بسم الله الرحمن الرحيم



دانشگاه صنعتی اصفهان

دانشکده علوم ریاضی

زبان برنامه نویسی تابعی Haskell برای روش های عددی

پروژه کارشناسی ریاضی کاربردی

سید احسان حسینی

استاد راهنما

دکتر رضا مختاری

۱۴۰۵

دانشگاه صنعتی اصفهان
دانشکده علوم ریاضی

پروژه کارشناسی ریاضی کاربردی آقای سید احسان حسینی

تحت عنوان

زبان برنامه نویسی تابعی Haskell برای روش های عددی

در تاریخ توسط کمیته تخصصی زیر مورد بررسی و تصویب نهایی قرار گرفت.

۱- اســـــــتاد راهنمای پروژه دکتر رضا مختاری

۲- اســـــــــــــــــتاد داور ۱

۳- اســـــــــــــــــتاد داور ۲

سرپرست تحصیلات تکمیلی دانشکده

به نام خداوند جان و خرد　　کز این برتر اندیشه بر نگذرد

خداوند نام و خداوند جای　　خداوند روزی ده رهنمای

خداوند کیوان و گردان سپهر　　فروزنده ماه و ناهید و مهر

ز نام و نشان و گمان برتر است　　نگارنده بر شده پیکر است

به بینندگان آفریننده را　　نبینی مرنجان دو بیننده را

نیابد بدو نیز اندیشه راه　　که او برتر از نام و از جایگاه

-فردوسی

در آغاز، بر خود لازم می دانم که از زحمات بی دریغ و راهنمایی های حکیمانه استاد راهنمای گرانقدر، جناب آقای دکتر رضا مختاری در راستای انجام این رساله تشکر و قدردانی نمایم. بی شک همراهی خاضعانه ایشان در طول دوره کارشناسی تا کارشناسی ارشد سزاوار سپاسی بیش از این است.

از خانم دکتر محدثه رمضانی که امر مشاوره این پایان نامه را به عهده داشتند، بسیار تشکرم. همچنین از داوران گرامی، آقای دکتر هادی روحانی و خانم دکتر مریم محمدی کمال سپاس و امتنان را دارم.

از پدر و مادر مهربان و عزیزم، برادرم ایلیا و خواهرم شهربانو که همواره مشوق و حامی من در سختی ها و دشواری های زندگی بوده اند، کمال تشکر را دارم.

در پایان از تمامی استادان و دوستان، از جمله آقایان دکتر سید محمود منجگانی و دکتر فرید بهرامی، که در طول سالیان تحصیلی همواره در کنار من بوده اند و تجربه های ارزنده ای به اینجانب آموخته اند، بسیار سپاس گزارم.

علی فروزنده، هفشجانی ۱۴۰۳/۰۶/۰۴

کلیه حقوق مالکیت مادی و معنوی مربوط به این پروژه متعلق به دانشگاه صنعتی اصفهان و پدیدآورندگان است. این حقوق توسط دانشگاه صنعتی اصفهان و براساس خط مشی مالکیت فکری این دانشگاه، ارزش‌گذاری و سهم‌بندی خواهد شد.

هر گونه بهره‌برداری از محتوا، نتایج یا اقدام برای تجاری‌سازی دستاوردهای این پروژه تنها با مجوز کتبی دانشگاه صنعتی اصفهان امکان‌پذیر است.

تقدیم به

فروزندگان راه تاریک علم؛

آنان که برای بیداری انسان از خواب غفلت، رویاها و زندگی خود را فدا کردند.

فهرست مطالب

فهرست مطالب هفت

پیشگفتار ده

۱ مقدمه ۱

۱.۱ مقدمه ای بر زبان های برنامه نویسی ۱

۲.۱ برخی پارادایم های رایج ۱

۳.۱ مقایسه دقیق تر پارادایم های تابعی و دستوری ۲

۴.۱ معرفی زبان Haskell ۲

۱.۴.۱ تاریخچه ۲

۲.۴.۱ ویژگی های مهم زبان ۳

۲ مبانی زبان Haskell ۵

۱.۲ مقدماتی از استاندارد زبان ۵

۲.۲ انواع ۷

۱.۲.۲ انواع عددی ۷

۲.۲.۲ انواع منطقی ۱۰

۳.۲.۲ انواع چندتایی های مرتب ۱۰

۴.۲.۲ انواع List ۱۰

۳.۲ کلاس های نوع (Type Classes) ۱۰

کتاب‌نامه ۱۱

واژه‌نامه فارسی به انگلیسی ۱۲

واژه‌نامه انگلیسی به فارسی ۱۵

فهرست نمادها ۲۰

فهرست کلمات اختصاری ۲۱

چکیده:

در این گزارش ابتدا به بررسی پارادایم های برنامه نویسی میپردازیم. سپس زبان Haskell را معرفی و مقدماتی بر این زبان ارایه میکنیم. ویژگی های زبان را بررسی میکنیم و نکاتی را هم درباره این زبان ذکر میکنیم. در نهایت به بررسی مثال هایی از کاربرد زبان Haskell در آنالیز عددی میپردازیم و مراجعی برای یادگیری عمیق تر زبان معرفی میکنیم.

پیشگفتار

در حل مسایل عددی زبان های برنامه نویسی نقش تایین کننده ای در نحوه پیاده سازی الگوریتم ها و روش های عددی دارند. پارادایم زبان، توانایی تشخیص و مدیریت خطاهای عددی یا باگ ها، میزان استفاده از حافظه و پردازشگر تحت تاثیر مستقیم زبان مورد استفاده هستند. اکثر زبان های مورد استفاده در محاسبات عددی از نوع imperative هستند. در این میان زبان های functional مثل Haskell کمتر استفاده میشوند. در این گزارش ابتدا مقدمه ای بر این زبان ارایه میکنیم و سپس ویژگی هایی از آن را بررسی میکنیم که میتوانند در حل مسایل کاربردی باشند و در کنار زبان هایی مانند MATLAB و C ابزار خوبی برای محققین باشد. زبان Haskell ویژگی های منحصر به فردی دارد که در آنرا تبدیل به ابزار مناسبی برای پژوهش میکند. در [؟] نویسندگان یک زبان Domain Specific (DSL) با Haskell میسازند تا یک زبان انتزاعی تر و ساده تر نسبت CUDA C++ به دست آید. در نهایت با بررسی این زبان و مقایسه عملکرد آن با کد CUDA C++ نتیجه میگرند که با کد های بسیار ساده تر میتوان به عملگردی مشابه به کد های CUDA C++ دست یافت. J. Karczmarczuk چند مقاله در زمینه مشتق گیری خودکار یا Automatic Differentiation (AD) دارد که در آنها از زبان Haskell برای پیاده سازی روش های خود استفاده کرده. در [؟] او از Lazy Evaulation و تکنیک های آن در تولید و استفاده از دنباله های طویل یا مشتقات عبارات عددی یک متغیره استفاده میکند.

فصل ۱

مقدمه

در این فصل زبان هارا از دیدگاه پارادایم بررسی میکنیم، اصطلاحات و تعاریف مورد نیاز بخش های بعدی را ارایه میکنیم.

۱.۱ مقدمه ای بر زبان های برنامه نویسی

واژه پارادایم تعریف دقیق و مشخصی ندارد اما راهی برای درک راحت تر و دسته بندی زبان های برنامه نویسی به دست میدهد. یک پارادایم برنامه نویسی یک شیوه یا نحوه نوشتن برنامه است. در هر زبانی شیوه هایی اسان تر و شیوه هایی سخت تر از دیگران است.

۲.۱ برخی پارادایم های رایج

پارادایم دستوری یا imperative رایج ترین است. در این نوع برنامه نویسی برنامه به صورت دنباله ای از دستورات صریح نوشته میشود. این دستورات در زمان اجرا تغییراتی در متغیر ها به وجود می آوردند که به عنوان وضعیت یا state شناخته میشوند. در زبان های C, Python, MATLAB بیشتر از این شیوه استفاده میشود. این زبان ها اصطلاحا Multi-paradigm هستند، به این معنی که میتوان در این زبان ها به شیوه های متعدد برنامه نوشت. در زبان های Python, MATLAB پارادایم های شی گرایی و تابعی هم قابل استفاده هستند. در پارادایم شی گرا یا object-oriented برنامه با تعریف اشیا یا object ها که هرکدام وضعیت خود و توابعی که آن وضعیت را تغییر میدهند

نوشته میشوند. ایده اصلی روش شی گرا این است که داده ها و توابع با هم جفت باشند. روش تابعی یا functional بر این ایده مبتنی است که برنامه دیگر وضعیت کلی نداشته باشند و تمام برنامه در چهاچوب توابع نوشته شده باشد. حالت خاص تر پارادایم تابعی، تابعی محض یا Purely Functional است که در آن برنامه به هیچ عنوان وضعیت ندارد، حلقه های تکرار وجود ندارند و توابع میتوانند با هم ترکیب شوند، به صورت ورودی به توابع داده شوند و یا خروجی توابع دیگر باشند. زبان مورد بررسی این گزارش، یعتی زبان Haskell از دسته زبان های تابعی محض است.

۳.۱ مقایسه دقیق تر پارادایم های تابعی و دستوری

در برنامه نویسی دستوری جریان کنترل (Control Flow) صریح است. هر دستور نشان میدهد چگونه پردازش و محاسبات صورت میگیرد. هر دستور بر وضعیت عمومی برنامه تاثیر میگذارد. کامپیوتر های دیجیتال طراحی شده اند تا کد ماشین (Machine Code) اجرا کنند که خود عموما به صورت دستوری نوشته میشود.
در برنامه نویسی تابعی برنامه ها توسط اعمال و ترکیب توابع ساخته میشوند. در این روش توابع مشابه توابع ریاضی درخت هایی از عبارات هستند که مجموعه ای از ورودی های قابل قبول را به مجموعه ای از خروجی ها میبرند. همانطور که اشاره شد حالت خاص روش تابعی، تابعی محض است. در این روش توابع برنامه تفاوت بزرگی با روش دستوری درند و آن در این است که همه توابع برنامه به ازای یک ورودی بک خروجی منحصر به فرد دارند. به عبارت دیگر خوش تعریف هستند. در زبان های دستوری توابع میتوانند خوش تعریف نباشند. برای مثال ورودی x به تابع f خروجی $y = f(x)$ میدهد اما با یک بار دیگر ورودی دادن x به تابع f میتوان خروجی $z = f(x)$ را دریافت کرد. در زبان هایی مانند C این رفتار ها بعضا باعث بروز باگ میشوند. باگ هایی که در زبان های تابعی محض به هیچ عنوان رخ نخواهند داد.

۴.۱ معرفی زبان Haskell

به منظور درک بهتر زبان در این بخش گریزی به تاریخچه زبان Haskell میرنیم و برخی کاربرد های آن را در صنعت فهرست میکنیم.

۱.۴.۱ تاریخچه

پس از انتشار زبان Miranda (یک زبان تابعی محض) در سال ۱۹۸۵ توسط شرکت .Research Software Ltd توجه به زبان های تنبل بسیار زیاد شد. تا سال ۱۹۸۵ بیش از ۱۰ زبان تابعی ساخته شده و مورد استفاده بودند

که هرکدام پیاده سازی ها و برداشت های متفاوتی در نحوه بهینه طراحی زبان ها داشتند. این موضوع در کنار تجاری بودن زبان Miranda باعث شد تا در کنفرانس Programmng Languages and Computer Architecture (FPCA '87) پیشنهاد تشکیل یک کمیته برای استاندارد سازی نحوه طراحی و ساخت زبان های مشابه طرح شد. وظیفه این کمیته طراحی زبانی مشترک بود که پایه ای برای تحقیقات آینده در حوزه طراحی زبان های تابعی باشد.
Haskell توسط یک گروه توسعه داده شد که تلاش میکرد از روش های بیشتر مطالعه شده استفاده کند. مفهوم Type Class ها برای اولین بار توسط Phillip Walder, Stephen Blott برای حل مشکلات تساوی ها و حساب انواع مختلف متغیر ها که زبان های موجود آن زمان را با مشکلاتی رو به رو کرده بود. زبان های برنامه نویسی موجود در آن زمان هرکدام راه متفاوتی را برای بررسی تساوی و حساب رفته بودند. مثلا زبان Standard ML عملگر جمع در کامپایلر برای متغیر های ممیز شناور و صحیح Float و Integer تعریف شده بود اما کاربر هیچ راهی برای تعریف عملگر جمع برای متغیر های دیگر نداشت. این موضوع برای بررسی تساوی میان دو متغیر هم صادق بود. Type Class ها راه حلی برای این مشکل بودند. به مرور زمان استاندارد زبان Haskell تکامل پیدا کرد و همچنان در حال توسعه است. استاندارد Haskell 2010 در حال حاظر تعریف رسمی این زبان است.

۲.۴.۱ ویژگی های مهم زبان

صفحه اصلی زبان ویژگی های زیر را علاوه بر Lazy Evaluation و Purely Functional را فهرست میکند. در اینجا به بررسی آنها میپردازیم.
این زبان ایستا گونه یا Statically Typed است به این معنی که نوع متغیر ها از قبل مشخص و در زمان اجرای برنامه ثابت است. زبان Haskell با زبان C که ایستا گونه است در این موضوع متفاوت است که نوع تمام عبارات هم در زمان کامپایل مشخص میشود. اگر کاپایلر تشخیص دهد در جایی از برنامه انواع با هم همخوانی ندارند برنامه را کامپایل نمیکند. زبان هسکل دارای ویژگی Type Inference یا استباط نوع است به این معنی که میتواند درباره نوع عبارات به صورت خودکار بدون دخالت کاربر استدلال کند و تشخیص دهد عبارت چه نوعی دارد.

```
main :: IO ()
main = do line :: String <- getLine
          print (parseDigit line)
  where parseDigit :: String -> Maybe Int
        parseDigit ((c :: Char) : _) =
          if isDigit c
             then Just (ord c - ord '0')
             else Nothing
```

عمل این برش کوتاه کد در اینجا مورد بحث نیست، اما در هرجه که نماد :: آمده نوع متغیر یا عبارت مشخص میشود. مثلا در parseDigit :: String -> Maybe Int نام parseDigit نوع تابعی که رشته دریافت و نوع Maybe Int

خروجی میدهد را میگیرد. ویژگی استنباط نوع باعث میشود در بسیاری از موارد دیگر نیازی به مشخص کردن نوع نباشد. کد زیر معادل کد بالاست.

```
main = do line <- getLine
          print (parseDigit line)
  where parseDigit (c : _) =
          if isDigit c
             then Just (ord c - ord '0')
             else Nothing
```

این زبان دارای کتابخانه سبکی برای برنامه نویسی موازی است. بررسی برنامه نویسی موازی در چهارچوب این گزارش نیست، به همین علت به آن نمیپردازیم و صرفا به ذکر این مطلب بسنده میکنیم که ویژگی های این زبان باعث میشوند برنامه نویسی موازی نقطه قوت آن باشد.

Haskell از استراتژی ارزیابی کندرو با Lazy Evaulation استفاده میکند. در این استراتژی، ارزیابی عبارات تا جای ممکن به تاخیر می افتند. این استراتژی موجب کاهش استفاده از حافظه در هر لحظه خواهد شد و با تکنیک هایی مثل Memoization میتوان به برنامه سرعت بخشید. در مقابل این استراتژی ترتیب برآورد شدن عبارات را بهم میریزد در نتیجه مدیریت استثناها و ورودی/خروجی ها با مشکلاتی رو به رو میشوند. علاوه بر آن میتواند منجر به نشت حافظه شود.

زبان Haskell دارای ۶۹۵۴ کتابخانه، موتور جستجوی Hackage برای کتابخانه ها و زبان، و ابزارهای Cabal و Stack برای ساخت و کامپایل برنامه هاست.

فصل ۲

مبانی زبان Haskell

در این فصل به آموزش مقدماتی زبان Haskell میپردازیم. در طول این فصل مباحثی از Haskell را بیان میکنیم که پیش نیازی از فصل های بعدی اند. این فصل دو قسمت است. در قسمت اول به آموزش زبان به صورت نظری میپردازیم و در بخش دوم از آموزه های بخش اول استفاده میکنیم تا تعدادی برنامه ساده بنویسیم. هر برنامه ای که در آن قسمت نوشته شود میتواند توسط خواننده نوشته و اجرا شود.

۱.۲ مقدماتی از استاندارد زبان

یک شناسه یا identifier ار یک حرف و بعد از آن هیچ یا چند حرف، زقم، _ ، علامت نقل قول مفرد ٰباید ساخته شود. شناسه ها به صورت لغوی به دو فضای نام namespace تقسیم میشوند: آنهایی که حرف اول آنها کوچک برای متغیر ها و آنهایی که حرف اول آنها بزرگ است برای سازنده ها constructor هستند. بزای مثال سازنده `Int` برای ساخت عداد صحیح با حرف اول بزرگ مشخص میشود. نام گذاری باقی بخش های برنامه قوانین از پیش تایین شده ای دارند. این قوانین به تفصیل در Haskell Report 2010 بحث شده اند. لازم به ذکر است که کامپایلر GHC که برای این گزارش استفاده شده، دارای توسیع هایی از این استاندارد است که تغییراتی جزیی در آن ایجاد کرده اند. تفاوت ها را میتوان در haskell.org در صفحه شناسه مربوط به هر نسخه کامپایلر دید.

نام گذاری در زبان Haskell به بزرگی و کوچکی حروف حساس است. به عنوان مثال `name` ، `naMe` و `Name` همگی شناسه های متفاوتی هستند. به طور خلاصه اسامی توابع و متغیر ها باید با حروف کوچک شروع شوند اما بعد از آن

برنامه نویس در انتخاب بقیه شناسه آزاد است.

در زبان Haskell میتوان از نوشتن نقطه ویرگول ; و آکولاد { } صرف نظر و از تورفتگی یا indentation برای چیدمان برنامه استفاده کرد. وقتی از چیدمان استفاده شود باید قوانینی رعایت شوند؛ در غیر این صورت کامپایلر با برگرداندن خطا متوقف میشود. به صورت غیر رسمی در جایی که از آکولاد و نقطه ویرگول صرف نظر شده باشد مشابه پایتون است. فرورفتگی باید برای عبارات یک چیدمان یکسان باشند. برای مثال کد زیر را ببینید. این دو برش کد معادل هستند.

```
module AStack( Stack, push, pop, top, size ) where
data Stack a = Empty
             | MkStack a (Stack a)

push :: a -> Stack a -> Stack a
push x s = MkStack x s

size :: Stack a -> Int
size s = length (stkToLst s)  where
           stkToLst  Empty         = []
           stkToLst (MkStack x s)  = x:xs where xs = stkToLst s

pop :: Stack a -> (a, Stack a)
pop (MkStack x s)
  = (x, case s of r -> i r where i x = x) -- (pop Empty) is an error

top :: Stack a -> a
top (MkStack x s) = x                    -- (top Empty) is an error
```

```
module AStack( Stack, push, pop, top, size ) where
{data Stack a = Empty
              | MkStack a (Stack a)

;push :: a -> Stack a -> Stack a
;push x s = MkStack x s

;size :: Stack a -> Int
;size s = length (stkToLst s)  where
           {stkToLst  Empty          = []
           ;stkToLst (MkStack x s)  = x:xs where {xs = stkToLst s

}};pop :: Stack a -> (a, Stack a)
;pop (MkStack x s)
  = (x, case s of {r -> i r where {i x = x}}) -- (pop Empty) is an error

;top :: Stack a -> a
;top (MkStack x s) = x                        -- (top Empty) is an error
}
```

توضیحات یا Comment ها همانطور که در بلوک های بالا میبینید مشابه زبان های برنامه نویسی دیگر رفتار میکنند. برای یک خط از دو خط تیره و برای بیش از آن از نماد های {- برای شروع بلوک توضیح و -} برای پایان آن استفاده میشود.

```
-- This is a Haskell comment
----- So is this
{- Also
        This
-}
```

۲.۲ انواع

در این قسمت به بررسی و معرفی انواع پایه ای داده در این زبان میپردازیم.

۱.۲.۲ انواع عددی

در زبان Haskell انواع عددی اساندارد پشتیبانی میشوند. در این زبان انواع متعددی از اعداد صحیح و ممیزی ارایه شده اند.

در زبان هسکل دو نوع از اعداد صحیح توسط استاندارد تعریف شده اند. `Int` و `Integer` نوع اول بازه محدودی از اعداد صحیح را پوشش میدهد. این بازه وابسته به تعداد بیت های اختصاص داده شده و وابسته به عللامت دار یا بی

علامت بودن عدد است. در هسکل به صورت انواع علامت دار و بی علامت اعداد صحیح برای حالت های ۸، ۱۶، ۳۲و ۶۴ بیت تعریف شده اند. نوع دوم اعداد صحیح علامت دار با اندازه بی کران هستند؛ به این معنی که عدد صحیح نوع Integer میتواند به هر اندازه بزرگ باشد.

اعداد ممیزی در زبان هسکل از ۴ نوع هستند. Float ، Double ، Rational و Scientific . اعداد Float از نوع Single Precision هستند. در مثال های زیر به بررسی نحوه تعریف این اعداد در محیط Glasgow Haskell Compiler interactive (GHCi) میپردازیم. این محیط در پیوست دو به تفصیل معرفی شده.

برای اجرای این دستورات در GHCi لازم است دستور زیر را ابتدا اجرا کنید تا کتابخانه های مورد نیاز فراخوانی شوند.

```
:m + Numeric GHC.Float Data.Int Data.Word
```

```
ghci> floatRadix (1 :: Double)
2
ghci> floatDigits (1 :: Double)
53
ghci> floatRange (1 :: Double)
(-1021, 1024)
ghci> floatRadix (1 :: Float)
2
ghci> floatDigits (1 :: Float)
24
ghci> floatRange (1 :: Float)
(-125, 128)
```

در بلوک بالا اطلاعات کلی را درباره انواع Float و Double میبینیم. توجه کنید که تابع floatRange بازه نمای عدد ممیز شناور را خروجی میدهد.

```
ghci> 0.1 + 0.2 :: Double
0.30000000000000004
ghci> 0.1 + 0.2 == (0.3 :: Double)
False
ghci> 0.1 + 0.2 :: Float
0.3
ghci> 0.1 + 0.2 == (0.3 :: Float)
True
```

در اینجا میبینیم که خطای نمایش اعداد ۰/۱ و ۰/۲ در نوع Double خروجی باقی میمانند اما همین خطاها در نوع Float در گردکردن پس از عمل جمع حذف میشوند. این با محیط Matlab متفاوت است. در محیط Matlab همین عمل جواب دقیق و درست ۰/۳ را تولید میکند. در نوشتن برنامه های عددی در زبان Haskell باید به این تفاوت ها توجه داشت. برای اینکه از رفتار عددی برنامه اطمینان حاصل کنیم بهتر است نوع هر عدد و بعضا خروجی هر عبارت را مشخص کنیم. در Haskell برای هر عبارتی که نوعش در برنامه مشخص نشده باشد توسط کامپایلر یک کلاس نوع

یا Type Class مشخص میشود. این کلاس ها در بخش سوم همین فصل به تفصیل بحث میشوند. به طور خلاصه انواع در این کلاس ها دسته بندی شده اند. این تعریف آنهارا ساده تر میکند. اعدادی اعشاری که در برنامه تولید میشوند در کلاس Fractional قرار میگیرند. این کلاس انواع `Float`, `Double`, `Rational` دارد. تا زمانی که کامپایلر اطمینان حاصل نکند که کدام نوع را باید برای یک عبارت در کلاس Fractional برگزیند عبارت "بدون نوع" مشخصی میماند. در حالتی که هیچ چیزی در برنامه برای کامپایلر نوع را مشخص نکند، کامپایلر از پیش فرض هایی برای انواع استفاده میکند. این پیش فرض ها در GHC's User Guide - 6.4 آمده اند. با توجه به این موضوع بدیهی است که در برنامه های عددی باید اطمینان حاصل کنیم که نوع درست انتخاب شده، بنابراین بهتر است خودمان آنها را مشخص کنیم.

```
ghci> toRational (0.1 :: Double)
3602879701896397 % 36028797018963968
ghci> decodeFloat (0.1 :: Double)
(7205759403792794,-56)
ghci> showHex (castDoubleToWord64 0.1) ""
"3fb999999999999a"
ghci> showHex (castFloatToWord32 0.1) ""
"3dcccccd"
```

نوع `Rational` اعداد را به صورت اعداد گویای تحویل ناپذیر یعنی نسبت دو عدد صحیح از نوع `Integer` ذخیره میکند. لازم به ذکر است که این نوع اعداد گویا هستند بنابراین نمیتوانند برای نمایش اعداد گنگ استفاده شوند اما تقریب های دقیق تر از `Double` را میتوان در این نوع ذخیره کرد. در بلوک بالا میبینیم که عدد `0.1 :: Double` در نوع `Rational` با کسر

$$\frac{3602879701896397}{3602879701896396}$$

ذخیره میشود. این کسر گویا شده ی نمایش دودیی عدد ۰/۱ در نوع `Double` است. باید در نظر داشت محدودیت تبدیل مبنا در نوع `Rational` همچنان جود دارد. در خطوط بعدی بلاک بالا نمایش های مبنای ۱۶ عدد ۰/۱ را در انواع `Double`, `Float` میبینیم. این ها دقیقا از قوانین IEEE 754 پیروی میکنند. بلاک زیر تاییدی بر این مطلب است.

```
ghci> (isIEEE (1 :: Double), floatRadix (1 :: Double), floatDigits (1 ::
    Double), floatRange (1 :: Double))
(True,2,53,(-1021,1024))
ghci> (floatDigits (1 :: Float), floatRange (1 :: Float))
(24,(-125,128))
ghci> encodeFloat 1 (-52) :: Double
2.220446049250313e-16
ghci> 1 + encodeFloat 1 (-53) == (1 :: Double)
True
```

در خروجی اول از چپ به راست ابتدا خروجی تابع `isIEEE` را میبینیم. این تابع به ازای ورودی هایی که از استاندارد

IEEE 754 پیروی میکنند خروجی میدهد. این برای بسیاری از انواع استاندارد زبان صحیح است اما ممکن است برای انواع ممیز شناور تعریف شده در کتابخانه ها درست نباشد. سپس مبنای اعداد را میبینیم که برابر ۲ است، تعداد بیت مانتیسا و بازه نمای عدد نوع `Double` . همه این مقادیر با استاندارد IEEE 754 همخوانی دارند. در خطوط بعدی تعداد بیت مانتیسای نوع `Float` بازه نمای آن، اپسیلون ماشین و در نهایت بررسی اینکه آیا عدد $\epsilon + ۱$ با یک برابر است یا خیر.

در بلاک های زیر به بررسی بیشتر حساب ممیز شناور میپردازیم.

```
sum (replicate 10 0.1) :: Double          -- 0.9999999999999999
sqrt 2 * sqrt 2 :: Double                 -- 2.0000000000000004
1e16 + 1 == (1e16 :: Double)              -- True (the 1 vanishes)
(1e16 + 1) + 1 == (1e16 + (1 + 1) :: Double)   -- False: addition isn't
    associative
```

۲.۲.۲ انواع منطقی

۳.۲.۲ انواع چندتایی های مرتب

۴.۲.۲ انواع List

۳.۲ کلاس های نوع (Type Classes)

کتاب‌نامه

واژه‌نامه فارسی به انگلیسی

ا

اساسی ..... Underlying
انتشاردهنده زمانی ..... Temporal propagator

پ

پیشرو ..... Marching

ت

توأم ..... Simultaneous
توزیع‌یافته ..... Distributed

چ

چندجمله‌ای‌های قطعه‌ای ..... Piecewise polynomials
چرخه ..... Cycle

د

درشت ..... Coarse
درون‌یاب سراسری ..... Global interpolation
درون‌یاب موضعی ..... Local interpolation
دوخطی ..... Bilinear

ر

رایانه خوشه‌ای ..... Cluster computer

ز

زبرینه اساسی ..... Essential supremum
زیرانتشار ..... Subdiffusion

س

سلسله مراتب ..... Hierarchy

ش

شبکه ..... Mesh, Grid

ظ

ظریف ..... Fine

ف

فراواقعی ..... Parareal
فضای آزمایش ..... Test Space
فضای آزمون ..... Trial Space

ق

قطری‌شونده توأم ..... Simultaneous diagonalization

ک

کاهش کلی ..... Total-reduction

کاهش تناوبی ..... Alternating-reduction

م

محدودیت ..... Restriction
محمل ..... Support
مقیاس‌پذیر ..... Scalabe

و

وادارنده ..... Coercive

ه

همروندسازی ..... Concurrency
هموار ..... Smooth

واژه‌نامه انگلیسی به فارسی

A

Affine ..... آفین
Analytic function ..... تابع تحلیلی

B

Boundary component ..... مولفه مرزی

C

Calculation ..... محاسبات
Center ..... مرکز
Change ..... تبدیل
Closed orbit ..... مدار بسته
Coefficient ..... ضریب
Compactness ..... فشردگی
Continuous ..... پیوسته
Contradiction ..... تناقض
Coordinates ..... مختصات
Corollary ..... نتیجه
Counted with multiplicities ..... با احتساب تکرار
Criteria ..... معیار

Critical period ..... دوره بحرانی
Curve ..... منحنی

D

Dehomogenized ..... غیرهمگن
Denominator ..... مخرج
Divid ..... تقسیم
Discriminant ..... مبین

E

Energy level ..... سطح انرژی
Explicit ..... صریح

F

Factor ..... عامل
First integral ..... انتگرال اول
Foliated ..... افراز (یا ورقه‌بندی) شده
Function ..... تابع

H

Hamiltonian system ..... دستگاه همیلتونی
Half-plane ..... نیم‌صفحه

I

Implicitly ..... به‌صورت ضمنی
Integrating factor ..... عامل انتگرال‌ساز
Interval ..... بازه
Invariant ..... ناوردا
Involution ..... برگردان
Isochronous ..... هم‌زمان-هم‌دوره

L

Lemma ..... لم

M

Multiplicities ..... چندگانگی‌ها
Maple ..... میپل
Mapping ..... نگاشت
Maximum ..... بیشینه
Minimum ..... کمینه
Monotone ..... یکنوا
Monotonicity ..... یکنوایی
Multiply ..... ضرب

N

Negative ..... منفی
Neighborhood ..... همسایگی
Notation ..... نماد

P

Parameter ..... پارامتر
Period annulus ..... طوق تناوبی
Period annuli ..... طوق‌های تناوبی
Period function ..... تابع دوره تناوب
Polynomial ..... چندجمله‌ای
Polycycle ..... چندسیکل
Potential system ..... دستگاه پتانسیل
Procedure ..... روش
Proof ..... اثبات
Proposition ..... گزاره
Punctured ..... محذوف

R

Regular ..... منظم
Rescaling ..... تغییرمقیاس
Resultant ..... برآیند
Reversible ..... برگشت‌پذیر
Root ..... ریشه

S

Sign ..... علامت
Statement ..... گزاره
Surrounding ..... حول

T

Tangent ..... مماس
Theorem ..... قضیه
Transformation ..... تبدیل
True ..... درست

U

Unique ..... یکتا

V

Variation method ..... روش تغییرات

W

Well defined ..... خوش‌تعریف

فهرست نمادها

نماد	مفهوم
$\mathbb{R}$	اعداد حقیقی
$\mathbb{R}^+$	اعداد حقیقی مثبت
$\mathbb{N}$	اعداد طبیعی
$\mathbb{O}$	اعداد فرد

فهرست کلمات اختصاری

اختصار	معنی	کلمه کامل
FEM	روش عناصر متناهی (المان محدود)	Finite elements method
FPDEs	معادلات با مشتقات جزئی کسری	Fractional partial differential equations
SFD	انتشار کسری-مکانی	Space-fractional diffusion
MGR	کاهش چندشبکه‌ای	Multigrid reduction
MGRIT	کاهش در زمان چندشبکه‌ای	Multigrid reduction-in-time

Multigrid reduction-in-time algorithms for solving fractional partial differential equations

Ali Forouzandeh Hafshejani
aforouzandeh@math.iut.ac.ir
August 31, 2024
Master of Science Thesis (in Farsi)
Departement of Mathematical Sciences
Isfahan University of Technology, Isfahan 84156-8311, Iran

Supervisor: Dr. Reza Mokhtari, mokhtari@iut.ac.ir
Advisor: Dr. Mohadese Ramezani, mohadeseh.ramezani@alumni.iut.ac.ir
2020 MSC: 65M55, 65M60, 65Y05.
Keywords: Fractional calculus, Space-fractional diffusion, FEM, Parallel-in-time, Multigrid reduction-in-time.

Abstract:
This M.Sc. thesis is based on the following paper

- Yue, X., Shu, S., Xu, X., Bu, W., & Pan, K. (2019). Parallel-in-time multigrid for space–time finite element approximations of two-dimensional space-fractional diffusion equations. *Comput. Math. Appl., 78*. 3471–3484.

Nowadays, temporal or spatial fractional partial differential equations (FPDEs) have found applications in real-world problems in science and engineering. This popularity stems from the fractional derivative's nonlocal property compared to the local nature of the integer-order derivative. One of the most important FPDEs is the space-fractional diffusion equation (SFD) which has applications in modeling anomalous diffusion, investigating subdiffusive phenomena, and describing chaotic dynamics. SFD over two-dimensional spaces is widely recognized as a key diffusion equation. It is derived from generalizing the spatial derivatives from integer order to fractional order within the partial differential equation. Since most SFD equations can not be solved analytically, various numerical methods, such as the finite difference method, local discontinuous Galerkin approach, and finite element method (FEM), have been proposed to achieve both high accuracy and efficiency. It is important to note that fractional derivatives use global information, while classical derivatives rely on local information. As a result, regardless of the discretization method used, significant computational effort is required due to the nonlocality introduced by fractional differential operators. Many researchers have worked on developing fast algorithms to address this challenge. In addition to these rapid solutions, parallel computing approaches, such as multigrid reduction in time (MGRIT), should also be considered potential techniques.
In this thesis, we begin by discussing fundamental concepts in functional analysis, including vector spaces, function spaces, and Sobolev spaces, as well as the principles of fractional calculus. We explain that fractional derivatives and integrals are foundational to fractional calculus, with the Ritz fractional derivative being particularly favored for applications in spatial domains. Next, we examine various fractional spaces, such as fractional Sobolev spaces, and investigate their properties. Additionally, we address spaces associated with FPDEs.
After that, we investigate the SFD problem with Dirichlet boundary conditions. We start by explaining the FEM and its properties. We construct the weak form of the SFD equation and apply space-time discretization, utilizing uniform spatial discretization and non-uniform temporal discretization. This process results in a large, sparse system of equations. To solve the SFD equation numerically, we represent the method as a time-marching loop, where a spatial linear system is solved at each time step. This time-marching loop acts as a one-step temporal method, similar to solving a lower bidiagonal block system over time. We also discuss various schemes for temporal parallelization and provide a brief historical context. One notable technique is MGRIT, which utilizes a multigrid reduction approach. The MGRIT offers two significant advantages: it minimizes interference with existing codes and allows optimal scalability. Subsequently, we employ a two-stage version of the MGRIT method to solve the lower bidiagonal system of the block unit and analyzing its convergence performance.
Finally, we implement a numerical example in MATLAB and XBraid and examine some tests. The results indicate that the method demonstrates adequate consistency and convergence for numerical solutions of such SFD equations, and it can be extended to solve some complicated FPDEs.

Isfahan University of Technology
Department of Mathematical Sciences

The Haskell Functional Programming Language for Numerical Methods

A Thesis
Submitted in Partial Fulfillment of the Requirements
for the Bachelor of Science (B. Sc.)

By
S. Ehsan Hoseini

Evaluated and approved by the thesis committee, on

1- Dr. Reza Mokhtari, Professor (Supervisor)

2- , (Examiner)

3- , (Examiner)

, (Department graduate coordinator)

Isfahan University of Technology
Department of Mathematical Sciences

Thesis Submitted for the Award of Bachelor of Science (B. Sc.) in Mathematics

The Haskell Functional Programming Language for Numerical Methods

S. Ehsan Hoseini

Supervisor: Dr. Reza Mokhtari